

PROYECTO FINAL

RetailX DataOps Lakehouse

Procesamiento de ventas con Docker, Apache Spark y Delta Lake

Curso	DataOps / Big Data / Arquitectura de Datos
Proyecto	Implementación de Lakehouse con arquitectura Medallion
Empresa simulada	RetailX
Estudiante	_____
Docente	_____
Fecha	_____

Índice

1. Introducción
2. Descripción del problema
3. Objetivos
4. Requerimientos
5. Arquitectura propuesta
6. Diseño del flujo DataOps
7. Estructura del proyecto
8. Implementación técnica
9. Ejecución paso a paso
10. Evidencias esperadas
11. Guion para presentación
12. Conclusiones
13. Recomendaciones
14. Anexos

1. Introducción

El presente proyecto desarrolla una solución DataOps tipo Lakehouse para RetailX, una empresa retail que necesita procesar grandes volúmenes de ventas. La solución utiliza Docker para facilitar la ejecución del ambiente, Apache Spark para procesamiento distribuido y Delta Lake para almacenar tablas confiables con soporte transaccional.

2. Descripción del problema

RetailX genera grandes cantidades de datos de ventas provenientes de distintos canales como tienda física, web, aplicación móvil y marketplace. El problema principal es organizar, limpiar y transformar esos datos para convertirlos en información útil para la toma de decisiones.

3. Objetivos

Objetivo general

Implementar un pipeline DataOps/Lakehouse que procese un millón de registros de ventas usando Docker, Apache Spark y Delta Lake bajo la arquitectura Medallion.

Objetivos específicos

- Generar un dataset sintético de ventas con 1,000,000 de registros.
- Implementar las capas Bronze, Silver y Gold.
- Limpiar y transformar datos usando PySpark.
- Crear tablas analíticas para KPIs y reportes.
- Documentar la arquitectura, ejecución y resultados obtenidos.

4. Requerimientos

Tipo	Requerimiento	Cumplimiento
Funcional	Generar dataset de ventas masivo.	Script 01_generate_dataset.py
Funcional	Procesar datos crudos en Bronze.	Script 02_bronze.py
Funcional	Limpiar y validar datos en Silver.	Script 03_silver.py
Funcional	Crear agregaciones de negocio en Gold.	Script 04_gold.py
No funcional	Ejecutar en ambiente portable.	Dockerfile y docker-compose.yml
No funcional	Facilitar auditoría y reproducción.	README y resultados en output/

5. Arquitectura propuesta

La arquitectura se basa en el patrón Medallion. Cada capa tiene una responsabilidad diferente para mejorar la calidad y disponibilidad de los datos.

Capa	Propósito	Salida
Bronze	Conservar datos crudos en formato Delta.	delta/bronze/sales
Silver	Limpiar datos, convertir tipos y calcular total_venta.	delta/silver/sales_clean
Gold	Crear indicadores y tablas agregadas para análisis.	delta/gold/*

Diagrama lógico del flujo:

CSV crudo -> Bronze Delta -> Silver Delta -> Gold Delta -> Consultas y reportes

6. Diseño del flujo DataOps

Ingesta

Se genera un archivo CSV con ventas simuladas. Cada registro incluye fecha, cliente, país, sucursal, producto, categoría, cantidad, precio, canal y método de pago.

Transformación

Spark lee el CSV y lo almacena en Delta Lake. Posteriormente limpia los datos, normaliza texto, convierte fechas y calcula el valor total de cada venta.

Explotación analítica

La capa Gold crea tablas resumidas: KPIs generales, ventas por categoría, ventas por país, ventas por mes y top productos.

7. Estructura del proyecto

```
retailx_dataops_lakehouse/
├── Dockerfile
├── docker-compose.yml
├── requirements.txt
├── README.md
├── jobs/
│   ├── 01_generate_dataset.py
│   ├── 02_bronze.py
│   ├── 03_silver.py
│   ├── 04_gold.py
│   ├── 05_queries.py
│   ├── run_pipeline.py
│   └── spark_session.py
├── data/
├── delta/
├── output/
└── docs/
```

8. Implementación técnica

Dataset

El dataset se genera mediante Python. La versión final debe ejecutarse con 1,000,000 registros, aunque para pruebas iniciales se recomienda usar 10,000 registros.

Spark y Delta Lake

La sesión de Spark se configura con extensiones de Delta Lake. Esto permite guardar y consultar tablas Delta en las carpetas de cada capa.

Indicadores generados

- Total de transacciones
- Ingresos totales
- Ticket promedio
- Clientes únicos
- Ventas por categoría
- Ventas por país
- Ventas por mes
- Top 10 productos por ventas

9. Ejecución paso a paso

Abrir Docker Desktop

Antes de ejecutar comandos, Docker debe estar iniciado.

Abrir PowerShell en la carpeta del proyecto

Ubicarse dentro de `retailx_dataops_lakehouse`.

Construir el contenedor

```
docker compose build
```

Levantar el contenedor

```
docker compose up -d
```

Entrar al contenedor

```
docker exec -it retailx_dataops_spark bash
```

Prueba rápida

```
python jobs/run_pipeline.py --rows 10000
```

Ejecución final

```
python jobs/run_pipeline.py --rows 1000000
```

10. Evidencias esperadas

1. Captura de `docker compose build` terminado correctamente.
2. Captura de `docker compose up -d` y `docker ps` mostrando el contenedor activo.

3. Captura del pipeline ejecutándose con 1,000,000 registros.
4. Captura de carpetas delta/bronze, delta/silver y delta/gold.
5. Captura de carpeta output con resultados CSV.
6. Captura de la consola mostrando KPIs generales y tablas Gold.

11. Guion para presentación

1. Presentar el problema: RetailX necesita procesar ventas masivas y convertir datos crudos en indicadores de negocio.
2. Explicar la arquitectura: Docker permite portabilidad, Spark procesa grandes volúmenes y Delta Lake almacena datos confiables.
3. Mostrar la arquitectura Medallion: Bronze recibe datos crudos, Silver limpia y Gold prepara reportes.
4. Ejecutar o mostrar evidencias del pipeline: dataset, capas Delta y resultados Gold.
5. Explicar los KPIs: ingresos totales, ticket promedio, ventas por categoría, ventas por país y top productos.
6. Cerrar con conclusiones: el sistema es reproducible, escalable y útil para analítica empresarial.

12. Conclusiones

- La arquitectura Lakehouse permite organizar datos de forma progresiva desde datos crudos hasta información analítica.
- Apache Spark facilita el procesamiento de grandes volúmenes de información.
- Delta Lake aporta confiabilidad y estructura para manejar tablas de datos.
- Docker permite que el proyecto sea ejecutable y portable en diferentes equipos.
- La capa Gold facilita la toma de decisiones mediante indicadores claros para negocio.

13. Recomendaciones

- Ejecutar primero con 10,000 registros para validar el ambiente.
- Usar 1,000,000 registros únicamente cuando Docker esté funcionando correctamente.
- Guardar capturas de cada etapa para evidenciar la ejecución.
- No borrar las carpetas delta/ ni output/ antes de tomar capturas.
- Explicar en la presentación el propósito de cada capa, no solo mostrar que el código corre.

14. Anexos

Comando principal

```
python jobs/run_pipeline.py --rows 1000000
```

Tablas Gold generadas

- kpis_generales
- ventas_por_categoria
- ventas_por_pais

- ventas_por_mes
- top_productos